



Pandas Functions Cheat Sheet

Comprehensive Guide to Data Manipulation in Python

Data Reading & Writing

Reading Data

```
import pandas as pd

# Reading CSV
df = pd.read_csv('data.csv', sep=',')

# Reading with specific options
df = pd.read_csv(
    'data.csv',
    sep=',',
    header=0,
    index_col=0,
    na_values=['NA', 'Missing']
)
```

read_csv()

read_excel()

read_json()

read_sql()

read_parquet()

read_pickle()

read_hdf()

read_sas()

read_feather()

read_gbq()

Writing Data

```
# Write to CSV
df.to_csv('output.csv', index=False)

# Write to Excel with sheets with
pd.ExcelWriter('output.xlsx') as writer:
    df1.to_excel(writer, sheet_name='Sheet1')
    df2.to_excel(writer, sheet_name='Sheet2')
```

to_csv()

to_excel()

to_json()

`to_sql()``to_parquet()``to_pickle()``to_hdf()``to_feather()``to_gbq()``to_html()`

Reading Data Tips

- Use `nrows` parameter to read only a portion of large files
- Set `low_memory=False` for mixed data types
- Use `usecols` to select specific columns
- Set `dtype` dictionary for memory optimization
- With CSVs, try `engine='c'` for speed or `engine='python'` for flexibility

Data Inspection

Basic Inspection

```
# Preview data
df.head(5) # First 5 rows
df.tail(3) # Last 3 rows

# Get information
df.info() # Column types and non-null values
df.shape # (rows, columns)
df.dtypes # Column data types
df.describe() # Statistical summary
```

`head()``tail()``info()``shape``dtypes``columns``index``describe()``memory_usage()``nunique()`

Missing Data

```
# Check for missing values
df.isna().sum() # Count missing values per column
df.isnull().any() # Check if any column has missing values

# Missing value percentage
(df.isnull().sum() / df.shape[0]) * 100
```

isna()

isnull()

notna()

notnull()

dropna()

fillna()

Unique Values

```
# Get unique values
df['column'].unique()

# Count unique values
df['column'].nunique()

# Value counts with normalization
df['column'].value_counts(normalize=True)
```

unique()

nunique()

value_counts()

duplicated()

drop_duplicates()

Data Manipulation

Filtering & Selection

```
# Boolean filtering
df[df['column'] > 100]

# Multiple conditions
df[(df['A'] > 0) & (df['B'] < 10)]

# isin filtering
df[df['category'].isin(['X', 'Y'])]
```

loc[]

iloc[]

at[]

iat[]

query()

filter()

isin()

between()

where()

mask()

Data Transformation

```
# Add new column
df['C'] = df['A'] + df['B']
```

```
# Apply function to column
df['log_val'] = df['value'].apply(lambda x: log(x))

# Replace values
df['category'].replace({'old': 'new'})
```

apply()

applymap()

map()

replace()

rename()

add_prefix()

add_suffix()

assign()

clip()

astype()

Reshaping

```
# Pivot table
pivot_df = df.pivot_table(
    values='value',
    index='row_var',
    columns='col_var',
    aggfunc='mean'
)

# Melt (unpivot)
melted_df = df.melt(
    id_vars=['id'],
    value_vars=['A', 'B'],
    var_name='variable',
    value_name='value'
)
```

pivot()

pivot_table()

melt()

stack()

unstack()

wide_to_long()

crosstab()

get_dummies()

explode()

factorize()



Aggregation & Grouping

Basic Aggregation

```
# Basic statistics
df['column'].mean()
df['column'].median()
```

```
df[['A', 'B']].sum()

# Multiple statistics at once
df.agg({
    'A': ['min', 'max', 'mean'],
    'B': ['median', 'std']
})
```

sum()

mean()

median()

min()

max()

count()

std()

var()

quantile()

agg()

GroupBy Operations

```
# Group by single column
df.groupby('category').mean()

# Group by multiple columns
df.groupby(['category', 'subcategory']).sum()

# Multiple aggregations
df.groupby('category').agg({
    'A': 'sum',
    'B': 'mean',
    'C': ['min', 'max']
})
```

groupby()

agg()

aggregate()

transform()

filter()

size()

count()

nth()

head()

tail()

Window Functions

```
# Rolling window
df['rolling_mean'] = df['value'].rolling(window=3).mean()

# Expanding window
df['cum_sum'] = df['value'].expanding().sum()

# Shifted values
df['prev_value'] = df['value'].shift(1)
df['next_value'] = df['value'].shift(-1)
```

rolling()

expanding()

shift()

diff()

pct_change()

cumsum()

cumprod()

cummax()

cummin()

rank()

GroupBy Tips

- Use `as_index=False` to keep group variables as columns
- Chain operations like `.groupby().agg().reset_index()`
- For custom aggregations, pass lambda functions
- Use `pd.NamedAgg` for clearer named aggregations
- Try `observed=True` for categorical data with missing categories

Merging & Joining

Concatenation

```
# Append rows
pd.concat([df1, df2], axis=0)

# Append columns
pd.concat([df1, df2], axis=1)

# With keys for hierarchical index
pd.concat([df1, df2], keys=['df1', 'df2'])
```

concat()

append()

Merging & Joining

```
# Basic merge on column
pd.merge(df1, df2, on='key')

# Merge with different key names
pd.merge(df1, df2, left_on='key1',
         right_on='key2')

# Different join types
pd.merge(df1, df2, how='left') # left, right, inner, outer
```

merge()

join()

merge_asof()

merge_ordered()

Combining Data

```
# Combine first non-null values
df1.combine_first(df2)

# Update values from another DataFrame
df1.update(df2)

# Combine using a function
df1.combine(df2, lambda s1, s2: s1 if s1.sum() > s2.sum() else s2)
```



@mdtahseneequbal